

Nexus Development Report - Registration & Backend Refactoring

Objective

Continue development of the Nexus authentication system by implementing user registration and restructuring the Flask backend into a modular architecture.

Registration Workflow

Workflow

User Opens Register Page
↓
Enter Username, Email, Password and Confirm Password
↓
Frontend Validation (Empty Fields & Password Match)
↓
JSON sent to Flask (/register)
↓
Flask checks SQLite
↓
Username Exists?
Yes → Return 'Username already exists'
No → Insert User into Database
↓
Registration Successful

Backend Architecture

Architecture Flow

Next.js Frontend
↓
Routes
↓
AuthService
↓
Database Layer
↓
SQLite Database

Project Structure

```
backend/  
app.py  
database/db.py  
models/user.py  
services/auth_service.py  
routes/home_routes.py  
routes/login_routes.py  
routes/register_routes.py
```

Database Refactoring

Earlier:

Login → `sqlite3.connect()`

Register → `sqlite3.connect()`

After Refactoring:

Login → `Database.connect()`

Register → `Database.connect()`

Database connectivity is now centralized in a single helper class.

Benefits

- Modular architecture
- Reduced code duplication
- Easier debugging
- Better scalability
- Ready for bcrypt, JWT and dashboard implementation

Conclusion

The registration module is functional, and the Flask backend has begun transitioning into a layered architecture using Models, Services, Routes and a Database layer.